BÀI THỰC HÀNH: Dự đoán giá nhà California bằng Neural Network (Regression)

1) Mục tiêu

Bạn sẽ xây dựng một mô hình **Multi-Layer Perceptron (MLP)** để dự đoán **giá nhà trung vị** (`median_house_value`) từ các đặc trưng địa lý & dân số.

Sau bài này, sinh viên phải:

- Làm đúng pipeline: **Load → EDA → Tiền xử lý → MLP → Train → Đánh giá → Cải tiến**
- Hiểu vì sao **chuẩn hóa dữ liệu** là bắt buộc cho MLP
- Biết xử lý **feature dạng categorical** bằng one-hot

---

2) Dataset

California Housing (Hands-On ML) – file CSV:

- Link CSV (raw): `https://raw.githubusercontent.com/ageron/handson-ml/master/datasets/housing/housing.csv`

Cột mục tiêu (target):

- `median_house_value` (giá nhà trung vị)

Một số cột đầu vào (features):

- `longitude`, `latitude`, `housing_median_age`, `total_rooms`, `total_bedrooms`, `population`, `households`, `median_income`, `ocean_proximity`

---

3) Yêu cầu bài tập

Sinh viên nộp:

1. Notebook chạy được từ đầu đến cuối
2. Báo cáo ngắn (1–2 trang hoặc markdown trong notebook) gồm:
    - Nhận xét EDA (ít nhất 3 nhận xét)
    - Giải thích các bước tiền xử lý

- Kết quả mô hình (RMSE/MAE/R^2) + nhận xét overfitting/underfitting
- Ít nhất 1 cải tiến mô hình và so sánh kết quả

---

4) Hướng dẫn thực hiện chi tiết

Bước 1 — Tải dữ liệu & xem nhanh

```
import pandas as pd

url = "https://raw.githubusercontent.com/ageron/handson-
ml/master/datasets/housing/housing.csv"
df = pd.read_csv(url)

df.head()
df.info()
df.describe()
```

Câu hỏi SV (trả lời trong notebook):

- Dataset có bao nhiêu dòng? Bao nhiêu cột?
- Có cột nào là categorical? Là cột nào?

Bước 2 — EDA tối thiểu

Yêu cầu SV làm tối thiểu:

- Kiểm tra missing values
- Xem phân phối target `median_house_value`
- Xem tương quan giữa `median_income` và target

Gợi ý code:

```
df.isna().sum().sort_values(ascending=False).head(10)
import matplotlib.pyplot as plt

plt.hist(df["median_house_value"], bins=50)
plt.title("Distribution of median_house_value")
plt.show()
df[["median_income","median_house_value"]].corr()
```

Câu hỏi SV:

- Target có bị "cắt ngưỡng" (capped) không? Dấu hiệu nào cho thấy điều đó?
- Feature nào có vẻ mạnh nhất dự đoán giá nhà?

Bước 3 — Tách X/y và chia train/val/test

```
from sklearn.model_selection import train_test_split

X = df.drop("median_house_value", axis=1)
y = df["median_house_value"]

X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.3,
random_state=42)
X_val, X_test, y_val, y_test   = train_test_split(X_temp, y_temp,
test_size=0.5, random_state=42)
```

Bước 4 — Tiền xử lý (Numeric + Categorical)

- Numeric: **StandardScaler**
- Categorical `ocean_proximity`: **OneHotEncoder**
- Gộp bằng `ColumnTransformer`

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.pipeline import Pipeline

num_cols = X_train.select_dtypes(exclude="object").columns
cat_cols = X_train.select_dtypes(include="object").columns

preprocess = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), num_cols),
        ("cat", OneHotEncoder(handle_unknown="ignore"), cat_cols),
    ]
)

X_train_p = preprocess.fit_transform(X_train)
X_val_p   = preprocess.transform(X_val)
X_test_p  = preprocess.transform(X_test)
```

Câu hỏi SV:

- Vì sao MLP cần chuẩn hóa numeric features?
- Vì sao phải one-hot cho `ocean_proximity`?

Bước 5 — Xây Neural Network (MLP) cho hồi quy

Mô hình baseline (đơn giản, dễ chạy):

```
import tensorflow as tf
```

```
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping

input_dim = X_train_p.shape[1]

model = Sequential([
    Dense(64, activation="relu", input_shape=(input_dim,)),
    Dense(32, activation="relu"),
    Dense(1)  # hồi quy: output linear
])

model.compile(
    optimizer="adam",
    loss="mse",
    metrics=[tf.keras.metrics.MeanAbsoluteError(name="mae")]
)

early_stop = EarlyStopping(monitor="val_loss", patience=10,
restore_best_weights=True)

history = model.fit(
    X_train_p, y_train,
    validation_data=(X_val_p, y_val),
    epochs=200,
    batch_size=64,
    callbacks=[early_stop],
    verbose=1
)
```

Bước 6 — Đánh giá mô hình (RMSE, MAE, R^2)

```
import numpy as np
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

y_pred = model.predict(X_test_p).reshape(-1)

rmse = np.sqrt(mean_squared_error(y_test, y_pred))
mae  = mean_absolute_error(y_test, y_pred)
r2   = r2_score(y_test, y_pred)

print("RMSE:", rmse)
print("MAE :", mae)
print("R2  :", r2)
```

Câu hỏi SV:

- RMSE và MAE khác nhau như thế nào? Khi nào ưu tiên RMSE?
- R^2 xấu thường do nguyên nhân gì?

Bước 7 — Trực quan hóa learning curves

```
import matplotlib.pyplot as plt

plt.plot(history.history["loss"], label="train_loss")
plt.plot(history.history["val_loss"], label="val_loss")
plt.legend()
plt.title("Learning curve (MSE)")
plt.show()
```

Câu hỏi SV:

- Mô hình đang overfit hay underfit? Dựa vào đâu?

---

5) Nhiệm vụ nâng cao

Chọn **ít nhất 1**:

1. Thêm **Dropout** hoặc **L2 regularization** và so sánh RMSE
2. Thử **tăng/giảm số layer**, số neuron → tìm cấu hình tốt hơn
3. Thử **log-transform target** (`log1p`) rồi dự đoán và `expm1` ngược lại
4. So sánh với một baseline truyền thống: **Linear Regression / RandomForest**

---

6) Tiêu chí chấm điểm

- Pipeline đúng + chạy ra kết quả: **40%**
- Tiền xử lý đúng (scaling + one-hot + split chuẩn): **25%**
- Đánh giá + biểu đồ learning curve + nhận xét: **25%**
- Bonus cải tiến + so sánh: **10%**